MP Online • Internship Project • 2025-26

Expense Tracker with Smart Insighter

Design Document

Prepared by: **ARYAN BHARTI**
Date: April 2026
MP Online Internship Project

1. Design Overview:

The design of the Expense Tracker with Smart Insighter focuses on building a modular, scalable, and maintainable system. The application follows a layered architecture, where each component is responsible for a specific functionality. This separation ensures that changes in one part of the system do not affect other components.

The design approach emphasizes:

- Clear separation between frontend, backend, and database
- Reusability of components
- Efficient data flow and communication
- Ease of future enhancements

2. High-Level Design (HLD):

The system is designed using a multi-layer architecture, consisting of the following components:

- Frontend (React) → Handles user interaction
- Backend (FastAPI) → Processes business logic
- Database (PostgreSQL) → Stores structured data
- AI/Logic Module → Provides intelligent insights Data Flow:

✞ User → Frontend → Backend API → AI Logic → Database → Backend → Frontend.

This design ensures:

- Loose coupling between components
- Scalable system structure
- Easy debugging and maintenance

3. Low-Level Design (LLD):

The Low-Level Design focuses on internal components and how they interact.

1. Frontend Components:
    - Login Component
    - Registration Component
    - Dashboard Component
    - Expense Form Component
    - Charts Component

Each component is designed as reusable and independent.

2. Backend Modules:
    - Routes Layer → Handles API endpoints
    - Service Layer → Contains business logic
    - Models Layer → Defines database structure
    - AI Service Module → Handles categorization, prediction, anomaly detection

3. Database Design:
    - Users Table
    - Expenses Table

(Relationship: One-to-Many)

4. Component Design:

- Frontend Design:
    - Built using React functional components
    - Uses state management (hooks) for UI updates
    - Communicates with backend via Axios

- Backend Design:
  - Built using FastAPI
  - Uses REST APIs for communication
  - Implements request validation using Pydantic

- Database Design:
  - Relational schema using PostgreSQL
  - Data stored in normalized tables
  - Uses ORM (SQLAlchemy) for interaction

5. API Design:

The backend exposes RESTful APIs for communication.
Sample APIs:

Method	Endpoint	Description
POST	/register	Register new user
POST	/login	Authenticate user
GET	/expenses	Fetch all expenses
POST	/expenses	Add new expense
PUT	/expenses/{id}	Update expense
DELETE	/expenses/{id}	Delete expense

6. Data Design:

The system handles structured financial data including:

- User information (name, email, password)
- Expense data (amount, category, description, date) Data is validated at:
- Frontend level (basic validation)
- Backend level (strict validation using schemas)

7. Security Design:

Security is implemented at multiple levels:

- Password hashing using bcrypt
- Protected routes in frontend
- Input validation to prevent invalid data

8. UI/UX Design Principles:

The user interface is designed with simplicity and usability in mind:

- Clean layout
- Minimal input fields
- Easy navigation
- Responsive design

Goal: Reduce user effort and improve experience.

9. Error Handling Design:

The system handles errors effectively:

- Invalid inputs → validation messages
- API errors → proper HTTP responses
- Authentication errors → access restrictions

This ensures system stability and better user guidance.

10. Scalability and Future Design:

The system is designed to support future enhancements:

- Integration with machine learning models
- Cloud deployment
- Mobile application support
- Real-time financial data integration

11. Design Constraints:

- Limited to rule-based AI (no ML model)
- No real-time bank API integration
- Designed for individual users

12. Conclusion of Design:

The system design ensures a balance between simplicity and functionality. By following a modular and layered approach, the application achieves scalability, maintainability, and performance efficiency. The design provides a strong foundation for future improvements and real-world deployment.